

Projeto 2 - Flight Manager

Algoritmos e Estruturas de Dados

Tomás Ferreira - up202002749

Gonalo Moreira - up202108804



src

 Airline.cpp

 Airline.h

 Airport.cpp

 Airport.h

 Flight.cpp

 Flight.h

 Graph.cpp

 Graph.h

 main.cpp

Classes

- ▶ Airline.cpp (airlines.csv)
- ▶ Flight.cpp (flights.csv)
- ▶ Airport.cpp (airports.csv)
- ▶ Graph.cpp (*.csv)

```
//read airlines.csv
while (std::getline( &: in2, &: token, dlm: '\n')) {
    std::stringstream iss( s: token);
    std::vector<std::string> temp;
    std::string tempstr;
    currentAirline = new Airline();

    while ((std::getline( &: iss, &: tempstr, dlm: ','))) {
        if (!tempstr.empty() && tempstr[tempstr.size() - 1] == '\r')
            tempstr.erase( pos: tempstr.size() - 1);
        temp.push_back(tempstr);
    }

    currentAirline->setCode(temp[0]);
    currentAirline->setName(temp[1]);
    currentAirline->setCallSign(temp[2]);
    currentAirline->setCountry(temp[3]);

    airlines[currentAirline->getCode()] = currentAirline;
}
```

```
//reads flights.csv
while (std::getline( &: in3, &: token, dlm: '\n')) {
    std::stringstream iss( s: token);
    std::vector<std::string> temp;
    std::string tempstr;
    Airport *origin, *destination;
    Airline *airline;

    while ((std::getline( &: iss, &: tempstr, dlm: ','))) {
        if (!tempstr.empty() && tempstr[tempstr.size() - 1] == '\r')
            tempstr.erase( pos: tempstr.size() - 1);
        temp.push_back(tempstr);
    }

    origin = airports[temp[0]];
    destination = airports[temp[1]];
    airline = airlines[temp[2]];
    origin->addFlight(destination, airline);
}
```

```
//reads airports.csv
while (std::getline( &: in1, &: token, dlm: '\n')) {
    std::stringstream iss( s: token);
    std::vector<std::string> temp;
    std::string tempstr;
    currentAirport = new Airport();

    while ((std::getline( &: iss, &: tempstr, dlm: ','))) {
        if (!tempstr.empty() && tempstr[tempstr.size() - 1] == '\r')
            tempstr.erase( pos: tempstr.size() - 1);
        temp.push_back(tempstr);
    }

    currentAirport->setCode(temp[0]);
    currentAirport->setName(temp[1]);
    currentAirport->setCity(temp[2]);
    currentAirport->setCountry(temp[3]);
    currentAirport->setLatitude(stod( str: temp[4]));
    currentAirport->setLongitude(stod( str: temp[5]));

    airports[currentAirport->getCode()] = currentAirport;
    n++;
}
```

ReadFiles()

Leitura de Dados

- ▶ Para realizar a leitura dos ficheiros .csv, utilizamos os métodos de `iss (istringstream)`.
- ▶ Decidimos guardar os dados lidos em vetores de strings temporárias que, posteriormente, foram utilizados para a construção dos objetos (`Airport`, `Airline`, `Flight`). Após cada objeto ser criado, foram inseridos na sua estrutura de dados respetiva. (`Airport` e `Airline` - `Unordered Map` na classe `Graph`), (`Flight` - `vector<flights>` na classe `Airport`).
- ▶ O grafo em si tem como nodes os aeroportos e cada aeroporto tem um vetor de `Flights` como edges. Dentro de cada `Flight` (edge) estão guardados parâmetros como o aeroporto de destino, a companhia aérea utilizada e a distância do voo.

Funcionalidades

- **readFiles()** - lê ficheiros de input (.csv). $O(n)$
- **calculateDistance()** - calcula a distância entre dois aeroportos. $O(1)$
- **bestTravel()** - retorna a melhor possibilidade de voos entre dois aeroportos ou cidades. $O(|V| + |E|)$
- **bestTravelAirport()** - **bestTravel()** se forem escolhidos aeroportos. $O(n)$
- **bestTravelCity()** - **bestTravel()** se forem escolhidas cidades. $O(n^2)$
- **getNumberOfFlightsforAirport()** - retorna o número de voos que saem de um aeroporto em específico. $O(1)$
- **getNumberOfAirlinesAirport()** - retorna o número de companhias aéreas que tem voos num aeroporto específico. $O(n)$

```
public:
    void readFiles(const std::string& file1, const std::string& file2, const std::string& file3);
    static double calculateDistance(Airport* a1, Airport* a2);
    Airport* bestTravel(Airport* origin, Airport* destination);
    void bestTravelAirport(Airport* origin, Airport* destination);
    void bestTravelCity(const string& origin, const string& destination);
    unordered_map<string, Airport*> getAirports();
    int getNumberOfFlightsForAirport(const string &airportCode);
    int getNumberOfAirlinesAirport(const string &airportCode);
    void listAirlines(const string &airportCode);
    void listFlights(const string &airportCode);
    int getNumberOfReachableCities(const string &airportCode);
    int getNumberOfReachableCountries(const string &airportCode);
    Airport *airlineBestTravel(Airport *origin, Airport *destination, const string &airlineCode);
    void oneAirlineBestTravel(Airport *origin, Airport *destination, const string &airlineCode);
    Airport *multipleAirlineBestTravel(Airport *origin, Airport *destination, const vector<string> &airlineCodes);
    void multipleAirlinesPrint(Airport *origin, Airport *destination, const vector<string> &airlineCodes);
    int multipleFlightsReachableCities(const string &airportCode, int numFlights);
    int multipleFlightsReachableCountries(const string &airportCode, int numFlights);
};
```

Funcionalidades

- **listAirlines()** - imprime o nome das companhias aéreas com voos nesse aeroporto. $O(n)$
- **listFlights()** - imprime os voos que saem desse aeroporto. $O(n)$
- **getNumberOfReachableCities()** - retorna o número de cidades que são possíveis de atingir com apenas um voo partindo de um aeroporto específico. $O(n)$
- **getNumberOfReachableCountries()** - retorna o número de países possíveis de atingir com apenas um voo partindo de um aeroporto em específico. $O(n)$
- **airlineBestTravel()** - retorna a melhor combinação de voos entre dois aeroportos usando apenas uma companhia aérea. $O(|V| + |E|)$
- **oneAirlineBestTravel()** - imprime o resultado da função airlineBestTravel(). $O(n)$

```
public:
    void readFiles(const std::string& file1, const std::string& file2, const std::string& file3);
    static double calculateDistance(Airport* a1, Airport* a2);
    Airport* bestTravel(Airport* origin, Airport* destination);
    void bestTravelAirport(Airport* origin, Airport* destination);
    void bestTravelCity(const string& origin, const string& destination);
    unordered_map<string, Airport*> getAirports();
    int getNumberOfFlightsForAirport(const string &airportCode);
    int getNumberOfAirlinesAirport(const string &airportCode);
    void listAirlines(const string &airportCode);
    void listFlights(const string &airportCode);
    int getNumberOfReachableCities(const string &airportCode);
    int getNumberOfReachableCountries(const string &airportCode);
    Airport *airlineBestTravel(Airport *origin, Airport *destination, const string &airlineCode);
    void oneAirlineBestTravel(Airport *origin, Airport *destination, const string &airlineCode);
    Airport *multipleAirlineBestTravel(Airport *origin, Airport *destination, const vector<string> &airlineCodes);
    void multipleAirlinesPrint(Airport *origin, Airport *destination, const vector<string> &airlineCodes);
    int multipleFlightsReachableCities(const string &airportCode, int numFlights);
    int multipleFlightsReachableCountries(const string &airportCode, int numFlights);
};
```

Funcionalidades

- **multipleAirlinesBestTravel()** - calcula o melhor caminho possível entre dois aeroportos usando x companhias aéreas (sendo x definido pelo utilizador). $O(|V| + |E|)$
- **multipleAirlinesPrint()** - imprime o resultado da função multipleAirlinesBestTravel(). $O(n)$
- **multipleFlightsReachableCities()** - retorna o número de cidades que são possíveis de atingir com x voos partindo de um aeroporto específico (sendo x definido pelo utilizador). $O(n^2)$
- **multipleFlightsReachableCountries()** - retorna o número de países possíveis de atingir com x voos partindo de um aeroporto em específico (sendo x definido pelo utilizador). $O(n^2)$

```
public:
    void readFiles(const std::string& file1, const std::string& file2, const std::string& file3);
    static double calculateDistance(Airport* a1, Airport* a2);
    Airport* bestTravel(Airport* origin, Airport* destination);
    void bestTravelAirport(Airport* origin, Airport* destination);
    void bestTravelCity(const string& origin, const string& destination);
    unordered_map<string, Airport*> getAirports();
    int getNumberOfFlightsForAirport(const string &airportCode);
    int getNumberOfAirlinesAirport(const string &airportCode);
    void listAirlines(const string &airportCode);
    void listFlights(const string &airportCode);
    int getNumberOfReachableCities(const string &airportCode);
    int getNumberOfReachableCountries(const string &airportCode);
    Airport *airlineBestTravel(Airport *origin, Airport *destination, const string &airlineCode);
    void oneAirlineBestTravel(Airport *origin, Airport *destination, const string &airlineCode);
    Airport *multipleAirlineBestTravel(Airport *origin, Airport *destination, const vector<string> &airlineCodes);
    void multipleAirlinesPrint(Airport *origin, Airport *destination, const vector<string> &airlineCodes);
    int multipleFlightsReachableCities(const string &airportCode, int numFlights);
    int multipleFlightsReachableCountries(const string &airportCode, int numFlights);
};
```


Interface (UI)

- Decidimos inserir vários tipos de Menus para as diferentes funcionalidades do programa melhorando assim a experiência do utilizador.

```
void showAirportInfoMenu() {
    cout << "\n";
    cout << "----- Menu -----" << endl;
    cout << "| 1- List of Flights                |" << endl;
    cout << "| 2- Number of Flights              |" << endl;
    cout << "| 3- List of Airlines                |" << endl;
    cout << "| 4- Number of Airlines              |" << endl;
    cout << "| 5- Number of Reachable countries   |" << endl;
    cout << "| 6- Number of Reachable cities      |" << endl;
    cout << "| 0- Go back to menu                 |" << endl;
    cout << "-----" << endl;
    cout << "Pick an option: ";
}

/**
 * Prints the sub-menu of the third choice
 */
void pickAirline() {
    cout << "\n";
    cout << "-----" << endl;
    cout << "| 1- Fly using one airline           |" << endl;
    cout << "| 2- Fly using several airlines      |" << endl;
    cout << "-----" << endl;
    cout << "Pick an option: ";
}
```

case 3:

```
cout << "===== " << endl;
cout << "||                ||" << endl;
cout << "|| Book your flight! ||" << endl;
cout << "||                ||" << endl;
cout << "===== " << endl;
```

```
void showMenu() {
    cout << "\n";
    cout << "----- Menu -----" << endl;
    cout << "| 1- Best flight possible            |" << endl;
    cout << "| 2- Airport info                    |" << endl;
    cout << "| 3- Book a flight                   |" << endl;
    cout << "| 0- Quit                            |" << endl;
    cout << "-----" << endl;
    cout << "Pick an option: ";
}

/**
 * Prints the sub-menu of the first choice
 */
void showBestFlightMenu() {
    cout << "\n";
    cout << "-----" << endl;
    cout << "| 1- Search by airport               |" << endl;
    cout << "| 2- Search by city                  |" << endl;
    cout << "-----" << endl;
    cout << "Pick an option: ";
}
```

```
void pickNumFlights() {
    cout << "\n";
    cout << "-----" << endl;
    cout << "| 1- Find within one flight          |" << endl;
    cout << "| 2- Find within multiple flights    |" << endl;
    cout << "-----" << endl;
    cout << "Pick an option: ";
}
```



```
/Users/tomasgrf/Desktop/AE02324_PRJ2_6117/cmake-build-debug/AE0project2
```

```
----- Menu -----  
| 1- Best flight possible |  
| 2- Airport info       |  
| 3- Book a flight      |  
| 0- Quit               |  
-----
```

```
Pick an option:
```

UX

```

Airport *Graph::bestTravel(Airport *origin, Airport *destination) {
    for (auto it : pair<...> : airports) it.second->setVisited(false);
    for (auto it : pair<...> : airports) it.second->setScales({});
    queue<Airport *> q; // queue of unvisited nodes
    q.push(v: origin);
    origin->setVisited(true);
    origin->setDistance(0);
    while (!q.empty()) { // while there are still unvisited nodes
        Airport *u = q.front();
        q.pop();
        for (auto e : Flight : u->getFlights()) {
            Airport *w = e.getDestination();
            vector<pair<Airport *, Airline *>> current;
            pair<Airport *, Airline *> curr;
            if (!w->isVisited()) {
                q.push(v: w);
                w->setVisited(true);
                double dist = calculateDistance(a1: w, a2: u);
                w->setDistance(u->getDistance() + dist);
                for (auto scale : pair<...> : u->getScales())
                    current.push_back(scale);
                curr.first = u;
                curr.second = e.getAirline();
                current.push_back(curr);
                w->setScales(current);
            }
            if (w == destination)
                return w;
        }
    }
    Airport *empty{};
    return empty;
}

```

Destaque para a melhor Funcionalidade

- Entre as variadas funções que implementamos, na nossa opinião, esta é das que está mais bem construída. Destacamos esta função bestTravel() que calcula o melhor caminho entre dois aeroportos usando BFS (Breadth-First-Search).

Principais Dificuldades e Avaliação

- ▶ Interpretação do enunciado e, na implementação das funções, a exigência de um grande domínio prático e teórico da matéria.
- ▶ Avaliação do grupo:
 1. Tomás Ferreira - 70%
 2. Gonçalo Moreira - 30%